

Introduction

- Pipelining is the process of arrangement of hardware elements of CPU such that its overall performance is increased.
 - The pipeline design provides a way to start a new task before an old task has been completed, which increases the speed
 - To achieve pipelining, divide the input task into a sequence of subtasks
 - Each subtasks can be processed independently on a different machine
 - Simultaneous execution of more than one instruction takes place in pipelined processor
-

Pipeline stages:

Pipeline Stages

Following are the 5 stages of pipeline with their respective operations:

- **Stage 1 (Instruction Fetch)**

In this stage the CPU reads instructions from the address in the memory whose value is present in the program counter.

- **Stage 2 (Instruction Decode)**

In this stage, instruction is decoded and the register file is accessed to get the values from the registers used in the instruction.

- **Stage 3 (Instruction Execute)**

In this stage, ALU operations are performed.

- **Stage 4 (Memory Access)**

In this stage, memory operands are read and written from/to the memory that is present in the instruction.

- **Stage 5 (Write Back)**

In this stage, computed/fetched value is written back to the register present in the instructions.

Pipelining

Let us consider the 'K' stage pipeline, with clock cycle time is 'Tp' and total 'n' tasks are to be executed. Time required to complete first task = kT_p

The remaining $(n-1)$ tasks emerge from the pipe at the rate of 1 task per clock cycle and they will be completed after a period equal to $(n-1)T_p$.

No. of clock cycles required = $K + (n-1)$, $K \rightarrow$ No. Of stages, $n \rightarrow$ No. of Instructions
If clock time = $T_p = 1\text{mSec}$, Total time required to execute the instructions = **$(K+(n-1))*T_p$**

Speedup = Time required in non pipeline / Time required in pipeline,
Speedup = $40/12 = 3.33$, After pipeline speed is increased by 3 times.

Efficiency = Utilisation = no. of boxes utilised / Total no. of boxes
 $= 40/60 = 2/3 = 0.66$

Efficiency = Utilisation = Speed up / No. of Stages = $3.33/5 = 2/3 = 0.66$

CPI(Clock per Instruction) = $K+(n-1)/n = 5+7/8 = 12/8 = 1.2$

Amdahl's Law

Amdahl's law relates the improvement of system's performance with the parts that did not perform well.

This law is often used in parallel computing to predict the theoretical speedup when using multiple processors

→ Amdahl's law is expressed mathematically as

$$\text{Speedup max} \leq \frac{1}{F + \frac{(1-F)}{P}}$$

$$S(P) \leq \frac{1}{F + \frac{(1-F)}{P}}$$

where p : no. of processors

F : sequential fraction of program

$1-F$: parallel fraction of program.

The microprocessor has integer and floating-point instructions. The floating-point instructions are enhanced and 3 times faster than before and integer instructions are unenhanced. If there are 20% of floating instructions in the program then find the overall speedup.

Solution: $F = 0.2$, $S = 3$, $\text{Speedup} = \frac{1}{(1-F) + \frac{F}{S}} = \frac{1}{0.8 + \frac{0.2}{3}} = 1.154$

Numerical

- A program having 10 instructions is executed on non-pipeline and pipeline processors. All instructions are of same length and having 4 pipeline stages and time required to each stage is 1ns.
- 1) Calculate time required to execute the program on non-pipeline and pipeline processor.
 - 2) Calculate speed-up.
- **Solution: Non pipeline processor: Cycle time = $4 \times 10 = 40$ nsec**
 - **Pipeline Processor(4 Stages): 2. Cycle time = $10 + 3 = 13$ nsec**
 - **Speedup = $40 / 13 = 3.08$**
-

Advantages and disadvantages of Pipelining

- Advantages of Pipeline: 1. The cycle time of the processor is reduced. 2. It increases the throughput of the system. 3. It makes the system reliable.
- Disadvantages of Pipelining: 1. The design of pipelined processor is complex and costly to manufacture. 2. The instruction latency(delay) is more.

Performance Parameters of Pipelined Execution

- The following parameters serve as criterion to estimate the performance of pipelined execution-
- Speed Up
- Efficiency
- Throughput
- **1. Speed Up:** It gives an idea of “**how much faster**” the pipelined execution is as compared to non-pipelined execution.
- It is calculated as-
- **2. Efficiency-** The efficiency of pipelined execution is calculated as-

$$\text{Speed Up (S)} = \frac{\text{Non-pipelined execution time}}{\text{Pipelined execution time}}$$

$$\text{Efficiency } (\eta) = \frac{\text{Speed Up}}{\text{Number of stages in Pipelined Architecture}}$$

OR

$$\text{Efficiency } (\eta) = \frac{\text{Number of boxes utilized in phase-time diagram}}{\text{Total number of boxes in phase-time diagram}}$$

Performance Parameters of Pipelined Execution

- **3. Throughput-** Throughput is defined as number of instructions executed per unit time.
- It is calculated as-

$$\text{Throughput} = \frac{\text{Number of instructions executed}}{\text{Total time taken}}$$

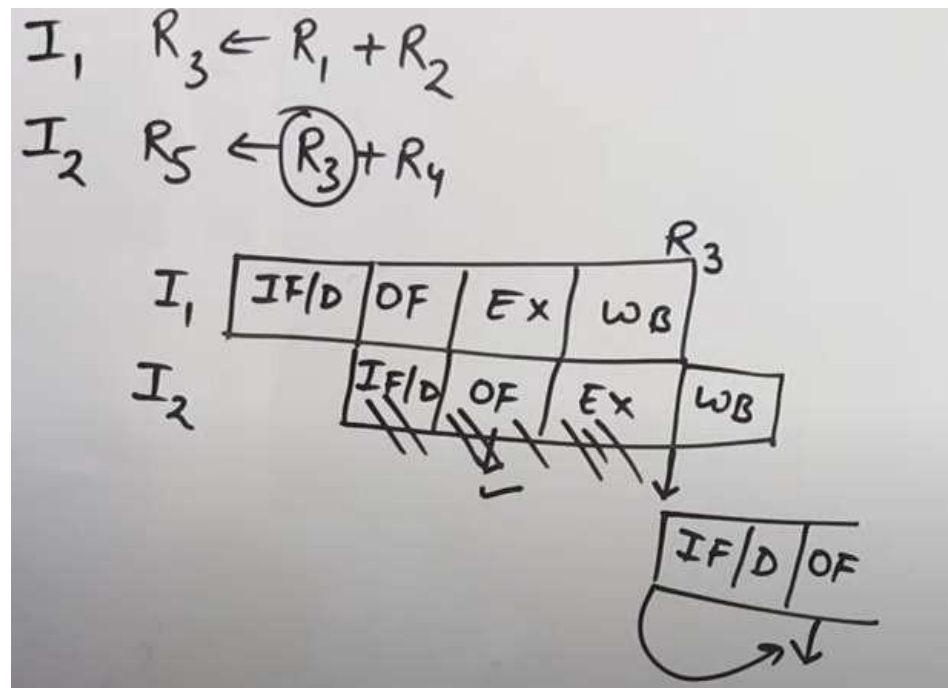
- CPI(Clock Per Instruction) should be equal to 1

Pipeline Hazards

- In pipeline, we assume CPI should be 1, i.e. in every clock cycle one instruction should get executed
 - Due to Hazards(problems) in pipelining, it is difficult to achieve $CPI = 1$
 - Delay occurred due to these hazards and therefore CPI is not equal to 1
 - Types of Hazards: 1. Data Hazards, 2. Structural hazards, 3. Control Hazards
 - Data Hazards: It occurs during READ WRITE operations on Data
 - Structural Hazards: When multiple instructions accessing the same resource(Processor, memory, registers, ALU)
 - Control Hazards: It occurs due to branch instructions
-

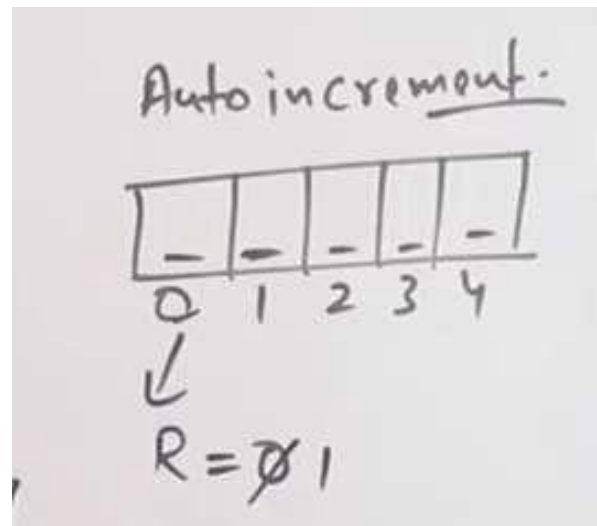
Data Hazards

- Data Hazards: RAW (Read After Write) Hazard
- I1 is executed first i.e. $R_3 \leftarrow R_1 + R_2$
- After the complete execution of I1, R3 is modified to new value i.e. after WB. But in the pipeline, I2 started execution and take the old value of R3 (which is not modified in I1) and will give the wrong result. To overcome the drawback, introduce stall and start I2 after 2,3 clock cycles



Data Hazards

- WAR(Write after Read) Hazard: This hazard does not appear in the normal execution of the instructions. Mostly RAW hazards appear in the program. WAR appear mostly in auto increment or auto decrement instructions. Because in auto INR or DCR, we first inr or dcr(Write) the content of register and then read. WAR hazard takes place in the instruction in which write operation is performed before the READ operation.



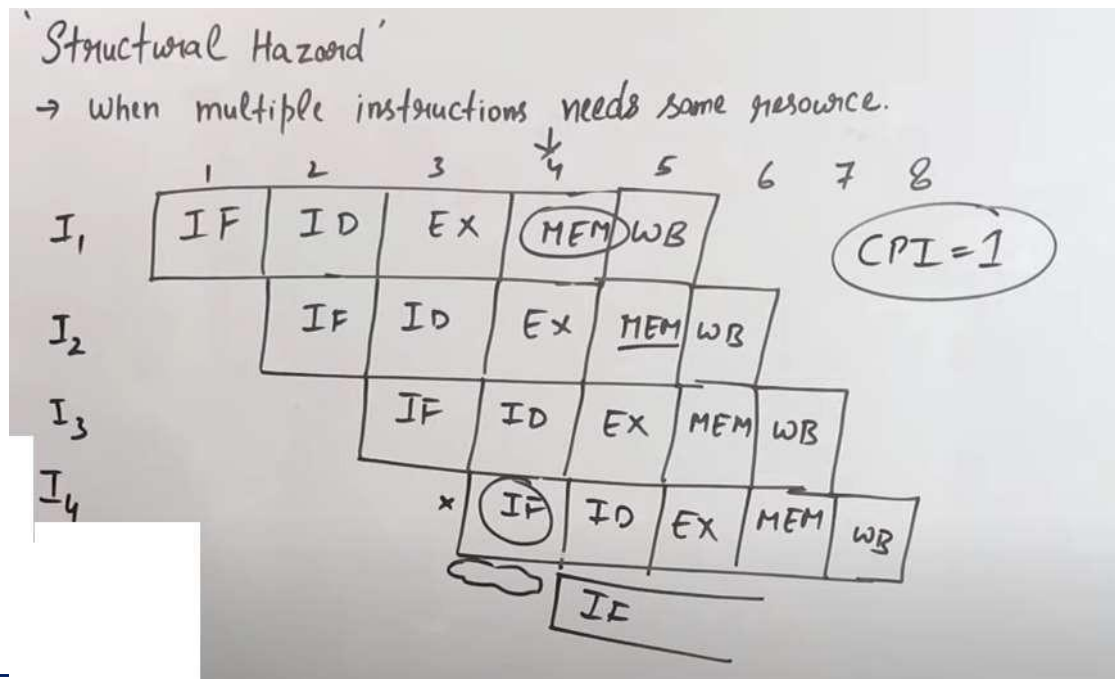
Data Hazards

- WAW(Write After Write) Hazard:
- From the example given below, instructions I1 and I2 are executing in a parallel manure. Actually, if I1 executed first then contents of $R3 = 200$ and after the execution of I2 the modified value of $R3 = 100$ but if the instruction I1 is delayed due to some reason then I2 executed first and $R3 = 100$, after the execution of I1, $R3 = 200$ which produces hazard
- This hazard will not occur in normal execution

The diagram shows two instructions, I1 and I2, both writing to register R3. Instruction I1 is $R_3 \leftarrow R_1 * R_2$ with values 200, 10, and 20 written above R_3 , R_1 , and R_2 respectively. Instruction I2 is $R_3 \leftarrow R_4 + R_5$ with values 100, 50, and 50 written below R_3 , R_4 , and R_5 respectively. The arrows indicate the data flow from the registers to the destination register R3.

Structural Hazards

- It occurs when multiple instructions use same resources (Processor, memory, registers, ALU)
- Instruction I1 in clock 4 using the main memory for data but at the same time instruction I4 using the main memory for instruction fetch and it causes structural hazards
- Solution: 1. Use different space for data and instructions in the main memory (Data Memory and Code Memory)
- 2. Use stall (No operation) in that clk, and start the instruction after 2 or 3 clk cycles, but it will increase no. of cycles
- 3. Use multiple resources, cost will increase
- 4. Use resources in pipeline but ckt. Will be very complex



Structural Hazards

Structural dependency

This dependency arises due to the resource conflict in the pipeline. A resource conflict is a situation when more than one instruction tries to access the same resource in the same cycle. A resource can be a register, memory, or ALU.

Example:

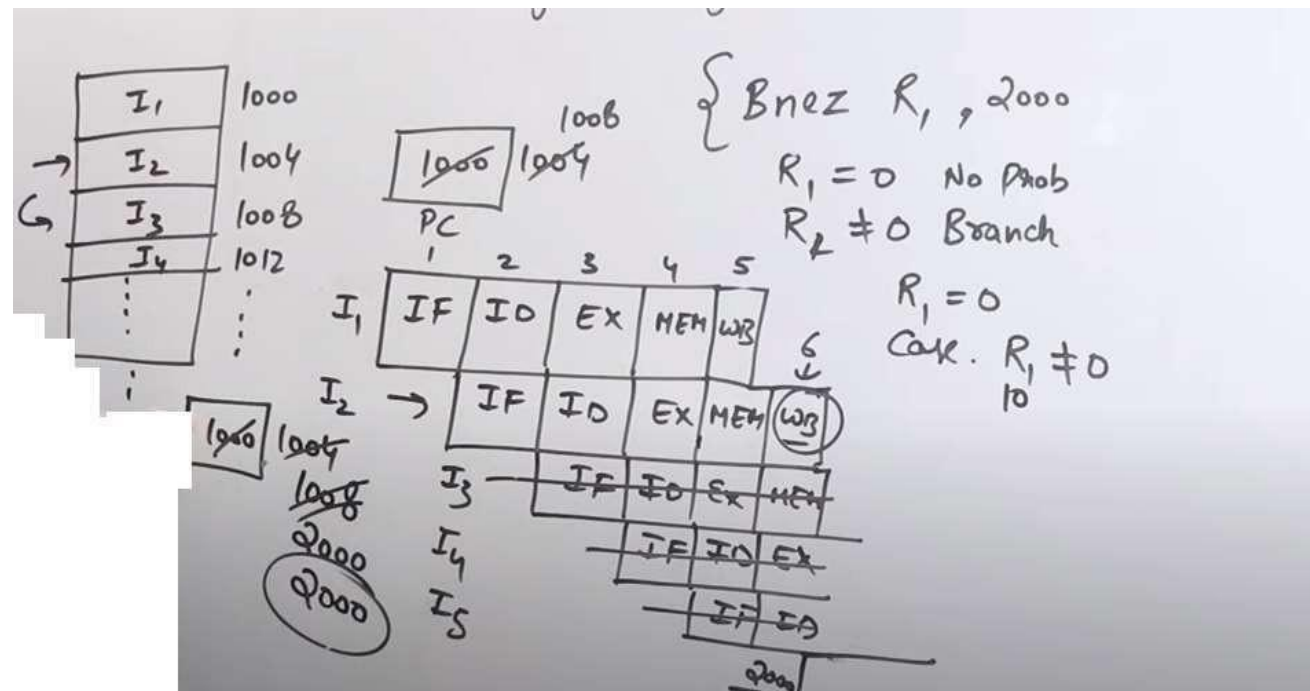
INSTRUCTION / CYCLE	1	2	3	4	5
I ₁	IF(Mem)	ID	EX	Mem	
I ₂		IF(Mem)	ID	EX	
I ₃			IF(Mem)	ID	EX
I ₄				IF(Mem)	ID

In the above scenario, in cycle 4, instructions I₁ and I₄ are trying to access same resource (Memory) which introduces a resource conflict. To avoid this problem, we have to keep the instruction on wait until the required resource (memory in our case) becomes available. This wait will introduce stalls in the pipeline as shown below:

CYCLE	1	2	3	4	5	6	7	8
I ₁	IF(Mem)	ID	EX	Mem	WB			
I ₂		IF(Mem)	ID	EX	Mem	WB		
I ₃			IF(Mem)	ID	EX	Mem	WB	
I ₄				-	-	-	IF(Mem)	

Control Hazards

- It occurs due to branch instructions eg. BRANCH, CALL, JUMP. All instructions who change the program counter leads to control hazards
- Instruction I1 is executed and if I2 is branch instruction, after the write back stage of I2, processor comes to know that It has to branch to the address 2000, given by PC, but till that time some stages of I3, I4 and I5 are already executed. So first, processor has to flush out the execution of I3, I4 and I5 and then execute the branch instruction but it will degrade the performance.
- Solution: Use stall when you come to know that it is branch instruction, but it will also degrade the performance



Control Hazards

Consider the following sequence of instructions in the program:

100: I₁

101: I₂ (JMP 250)

102: I₃ 250: BI₁

Expected output: I₁ -> I₂ -> BI₁

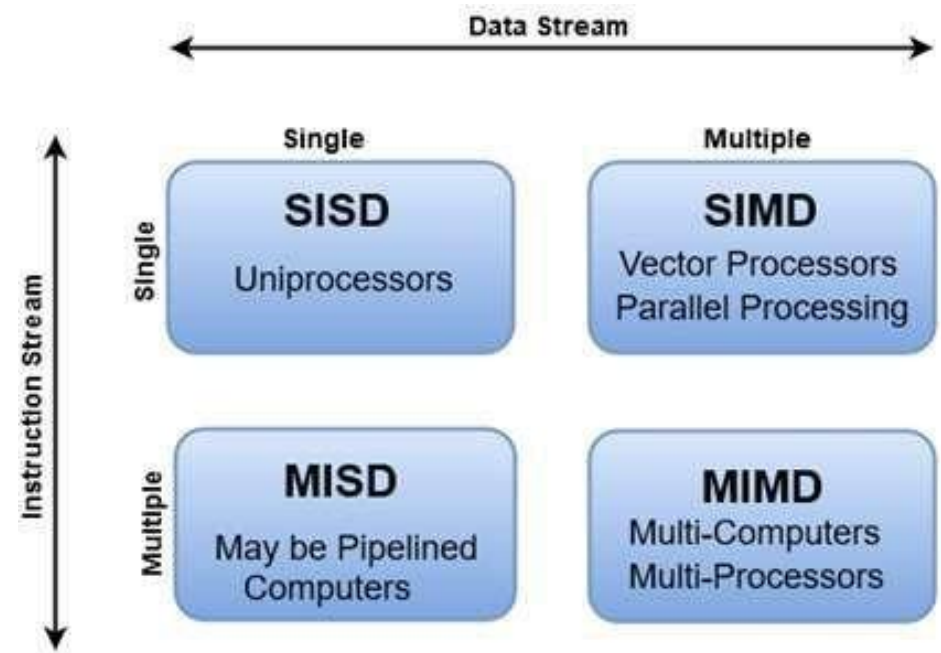
NOTE: Generally, the target address of the JMP instruction is known after ID stage only.

INSTRUCTION/ CYCLE	1	2	3	4	5	6
I ₁	IF	ID	EX	MEM	WB	
I ₂		IF	ID (PC:250)	EX	Mem	WB
I ₃			IF	ID	EX	Mem
BI ₁				IF	ID	EX

Output Sequence: I₁ -> I₂ -> I₃ -> BI₁

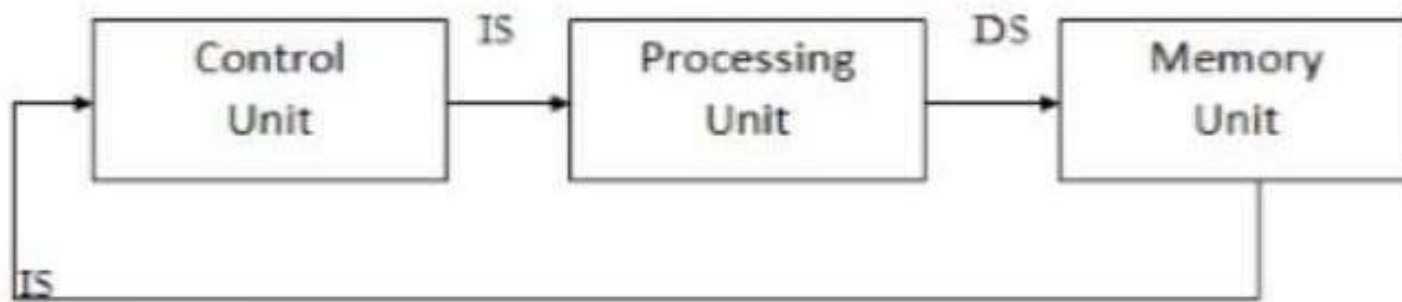
Flynn's Classification of computers

- Flynn's Classification: The most popular classification of computer architecture is defined by Flynn in 1966.
 - Classification based on stream of information
 - Two types of information flow into a processor: 1. An Instruction stream (sequence of instructions executed by CPU), 2. A Data stream (Data exchanged between the memory and CPU)
 - Instruction and data stream may be single or multiple. On this basis Flynn has given a classification of processor organization
-
- They are classified into 4 types:
 - SISD (Single Instruction Single Data)
 - SIMD (Single Instruction Multiple Data)
 - MISD (Multiple Instruction Single Data)
 - MIMD (Multiple Instruction Multiple Data)
 - Instructions and data are fetched from memory and these are decoded by Central Unit and send to Processing Unit for further Execution.



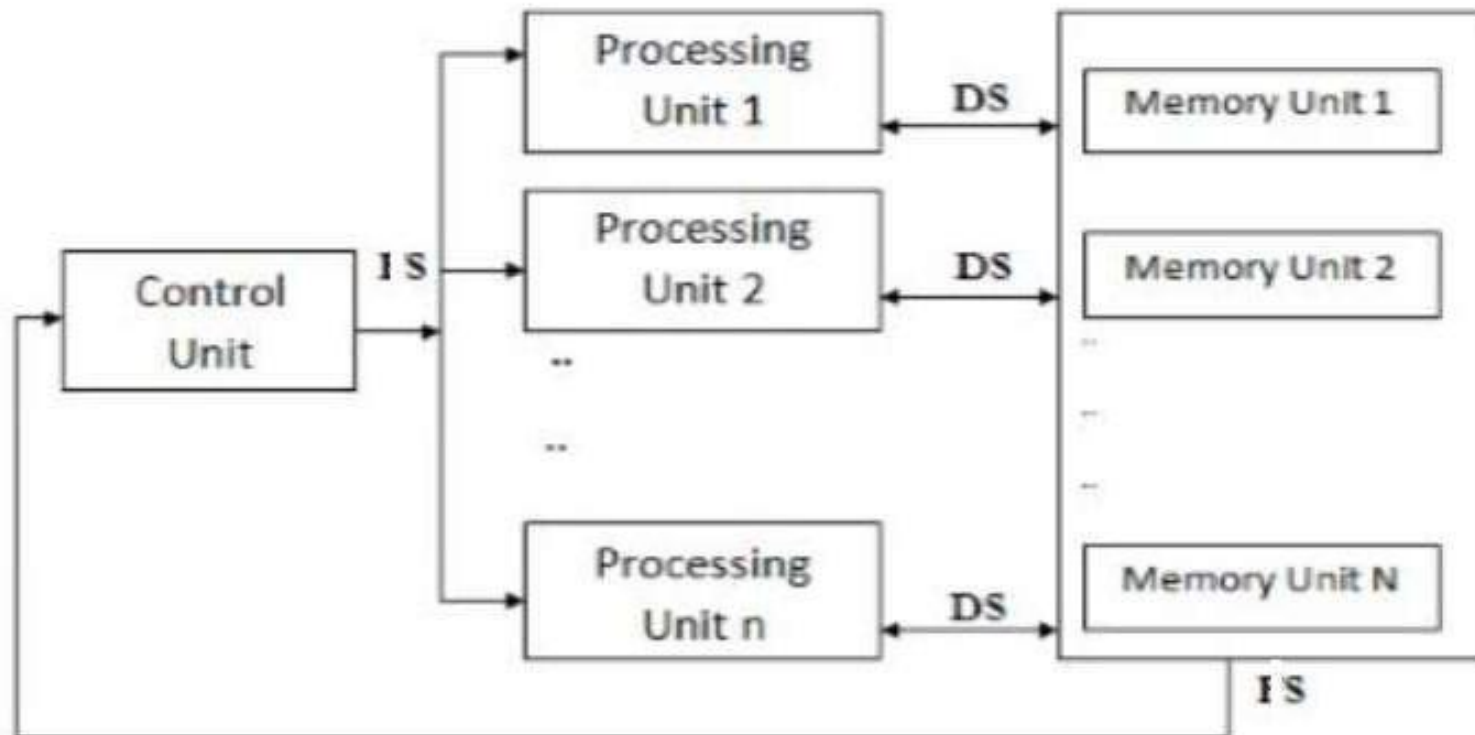
SISD(Single Instruction Single Data)

- A SISD computing system is a uniprocessor machine which is capable of executing a single instruction, operating on a single data stream. In SISD, machine instructions are processed in a sequential manner and computers adopting this model are popularly called sequential computers. Most conventional computers have SISD architecture. Eg. Von Neumann computer. Eg. $Z = \sin(x)$



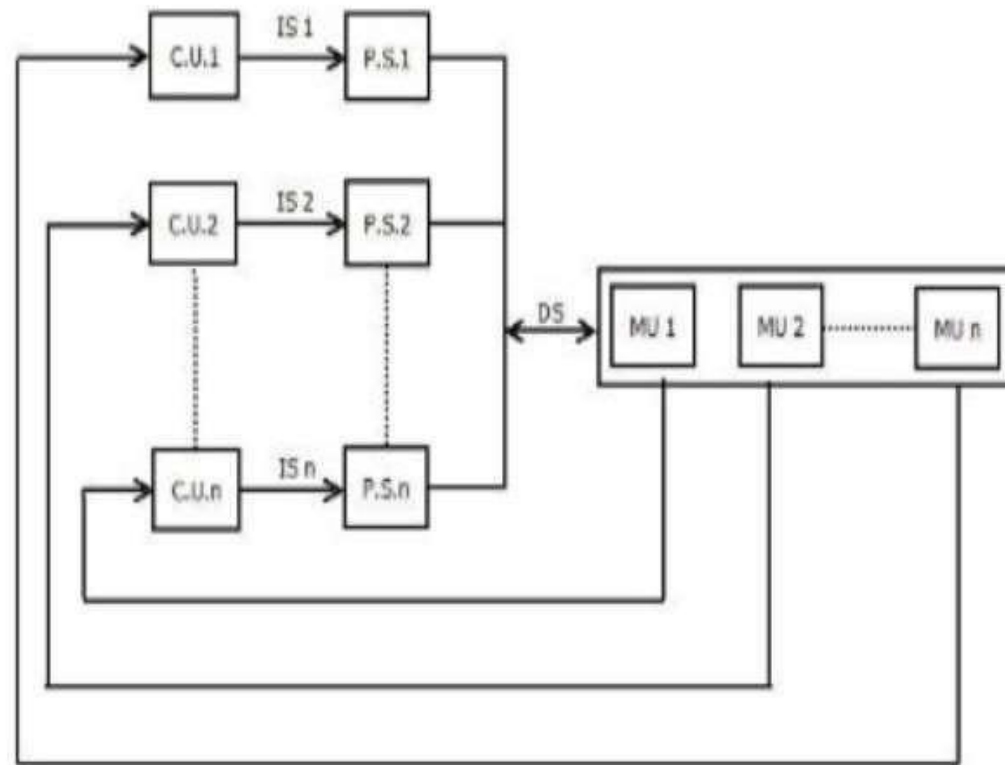
SIMD (Single Instruction Multiple Data)

- In this, there are multiple processing units which are controlled by Control Unit. All the Processing Units receives same I.S from Control Unit and operates on different data items from different DS. It is best suited for image processing application. IBM9000 and Hitachi computers using this technique. $Z = (x) + (y)$



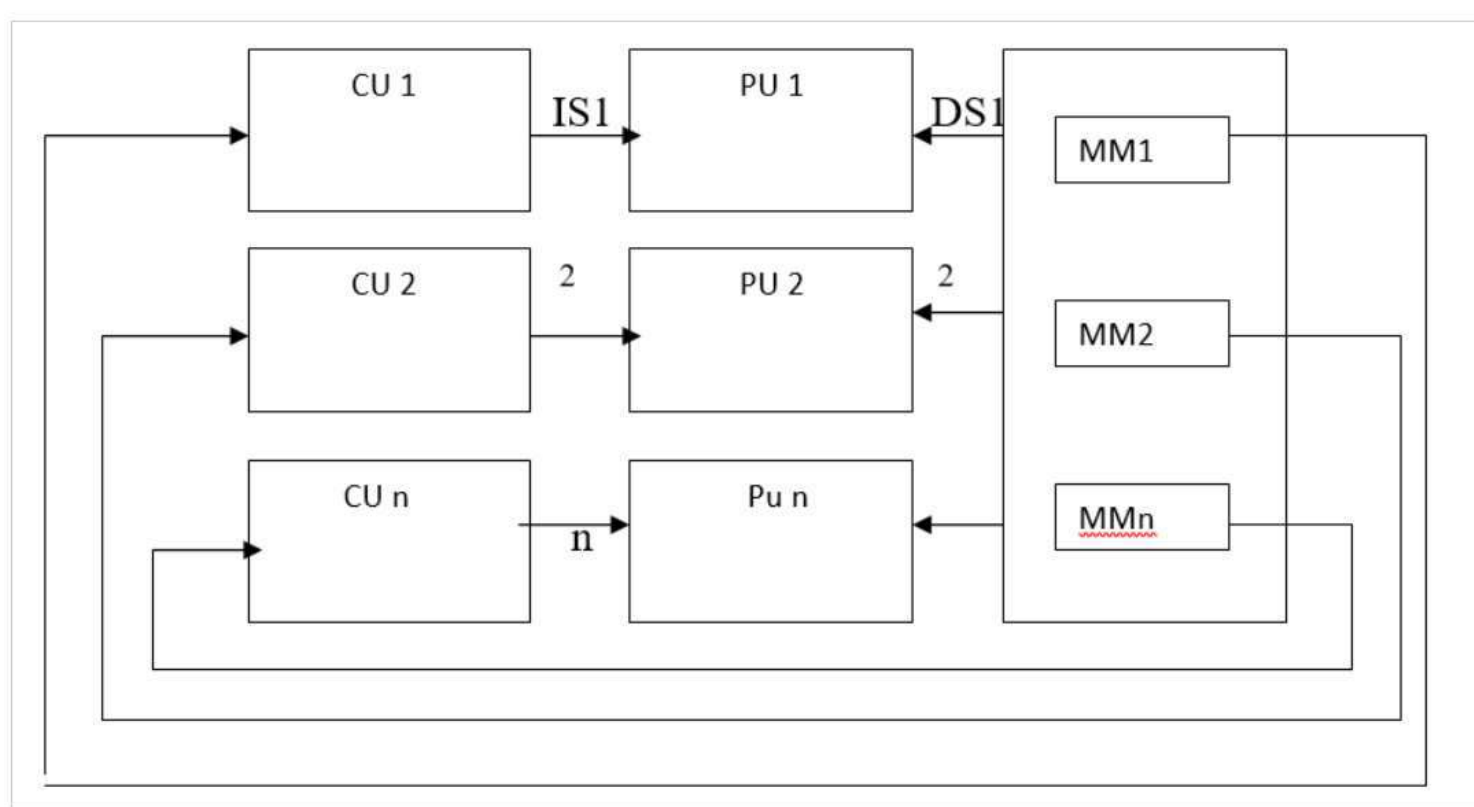
MISD (Multiple Instruction Single Data)

- In MISD, there are n processor units & each receive different instructions from different Control Units & operates on same data stream.
- Eg. 1. $Z = \sin(x) + \cos(x) + \tan(x)$
- 2. Multiple cryptography algorithm to crack a single coded message



MIMD(Multiple Instruction Multiple Data)

- A MIMD system is a multiprocessor machine that is capable of executing multiple instructions over multiple data streams. Each processing element has a separate instruction stream and data stream. $Z = \sin(x) + \cos(y) + \tan(z)$



Bus Architecture

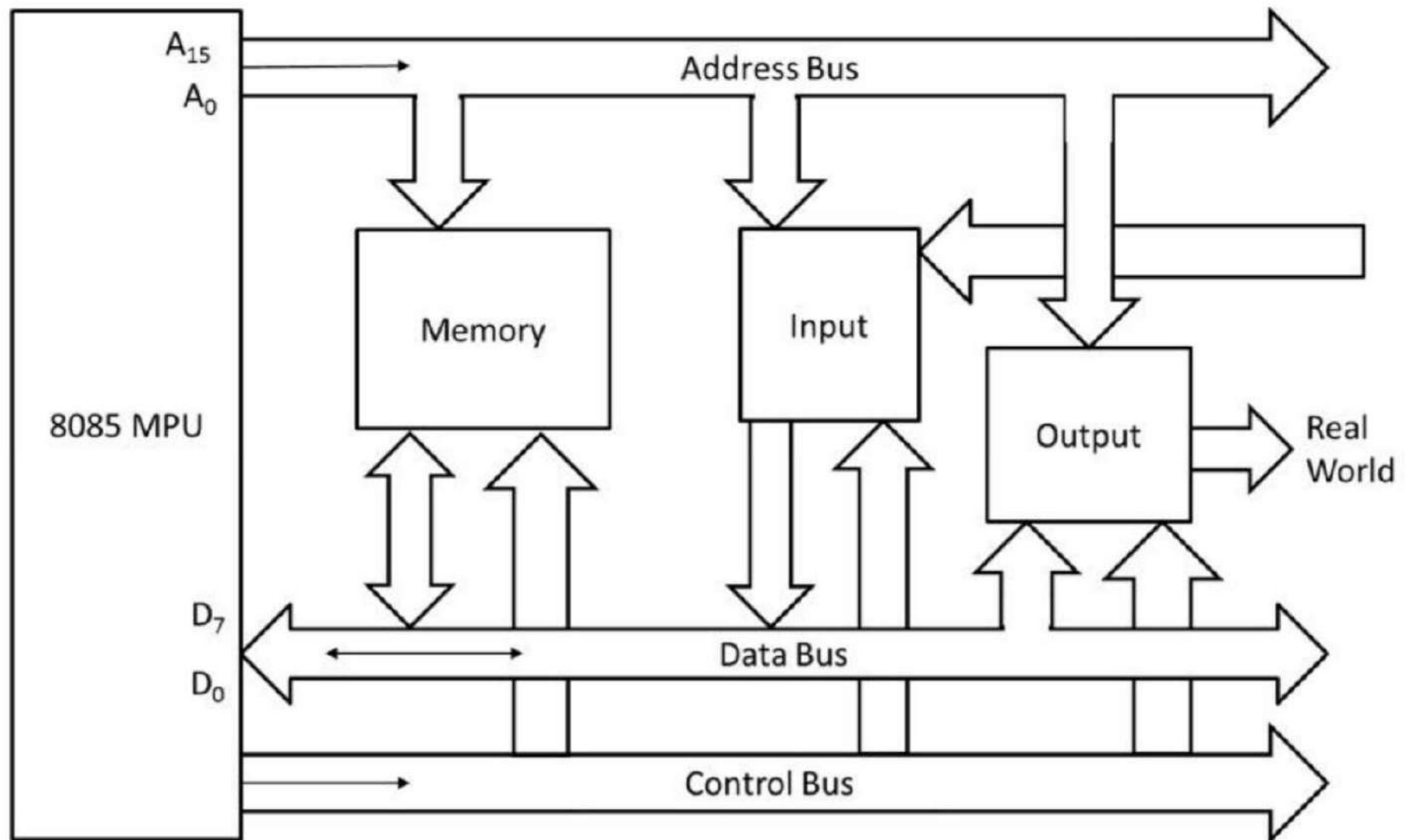


Figure: Bus Organization of 8085.